Lab 2 : MultiCycle Performance

Authors:
Connor Wilson

Professor:
Dr. Maria Pantoja

CPE333
Summer 2026
July 6, 2026

Contents

0.1 Matrix Multiplication Performance 2
0.1.1 Procedure 2
0.1.2 Results 2
0.2 Testall Performance 7
0.2.1 Procedure 7
0.2.2 Results 7
0.3 Notes 10
0.3.1 RARS 10

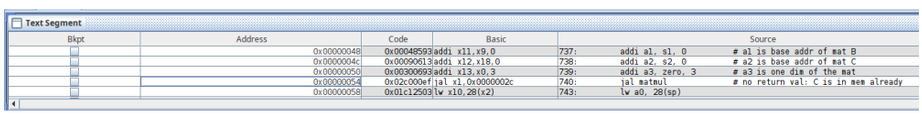
0.1 Matrix Multiplication Performance

0.1.1 Procedure

1. For each matrix test case (3x3, 10x10, 16x16, 50x50) . . .
2. Compile the matrix multiplication program in the OTTER-memory mapped version of RARS. The RARS program should compile but not run any instructions of assembly. The machine code file needs to be saved as a .mem
3. Load the mem file into the multicycle OTTER in Vivado and simulate.
4. Capture the waveforms for the matrix multiplication to prove the correct execution.
5. Provide CPU execution time , Power, and Area utilization.

0.1.2 Results

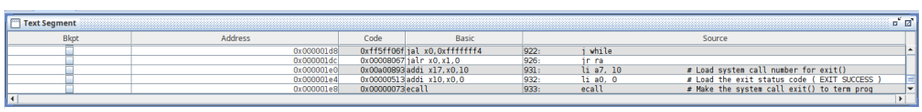
All of the RARS matrix multiply compiles start and end with the same instruction address. For timing purposes, the hex address $0x54$ for start and $0x1E0$ will be used for the stop.



Text Segment

Bkpt	Address	Code	Basic		Source
	0x00000048	0x00048593	addi x11,x9,0	737: addi a1, s1, 0	# a1 is base addr of mat B
	0x0000004c	0x00090613	addi x12,x18,0	738: addi a2, s2, 0	# a2 is base addr of mat C
	0x00000050	0x00300693	addi x13,x0,3	739: addi a3, zero, 3	# a3 is one dim of the mat
	0x00000054	0x02c000ef	jal x1,0x0000002c	740: jal matmul	# no return val: C is in mem already
	0x00000058	0x01c12503	lw x10,28(x2)	743: lw a0, 28(sp)	

Figure 1: MatrixMult.asm Start Codes for PC



Text Segment

Bkpt	Address	Code	Basic		Source
	0x000001d8	0xff5ff06f	jal x0,0xffffffff4	922: j while	
	0x000001dc	0x00008067	jalr x0,x1,0	926: jr ra	
	0x000001e0	0x00a00893	addi x17,x0,10	931: li a7, 10	# Load system call number for exit()
	0x000001e4	0x00000513	addi x10,x0,0	932: li a0, 0	# Load the exit status code (EXIT_SUCCESS)
	0x000001e8	0x00000073	ecall	933: ecall	# Make the system call exit() to term prog

Figure 2: MatrixMult.asm Exit Codes for PC

TC 3x3

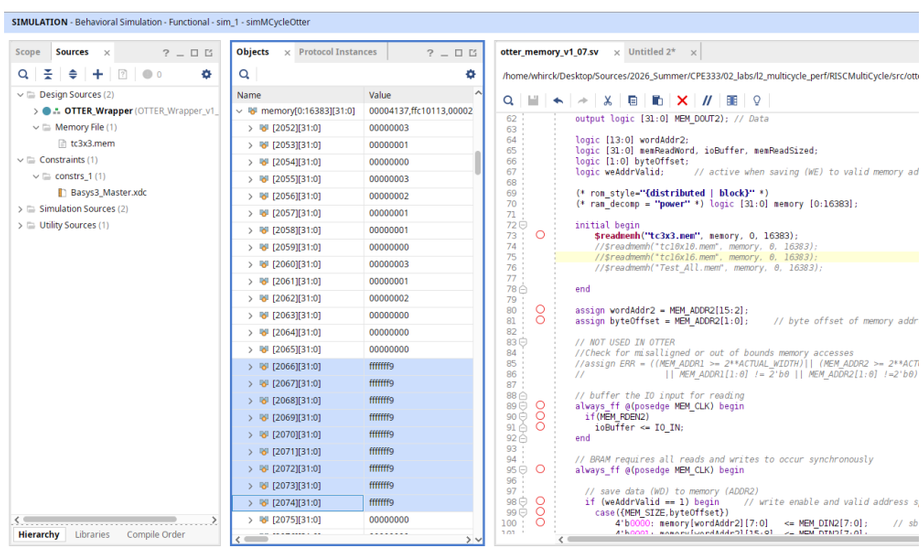


Figure 3: Results Matrix "C" before instruction execution.

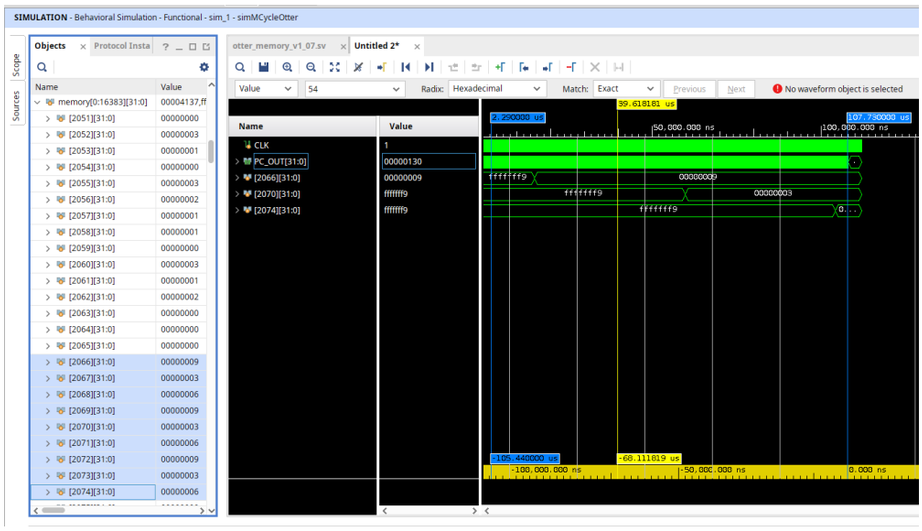


Figure 4: Waveform and Timing of TC 3x3 mat

$$
\begin{aligned}
t_{execution} &= t_{end} - t_{start} \\
&= 107.730\mu s - 2.290\mu s \\
&= 105.44\mu s
\end{aligned}
$$

$$
\begin{aligned}
n_{CLK_Cycles} &= \frac{t_{execution}}{T_{CLK}} \\
&= \frac{105.44\mu s}{10ns} \\
&= 10,544
\end{aligned}
$$

TC 10x10

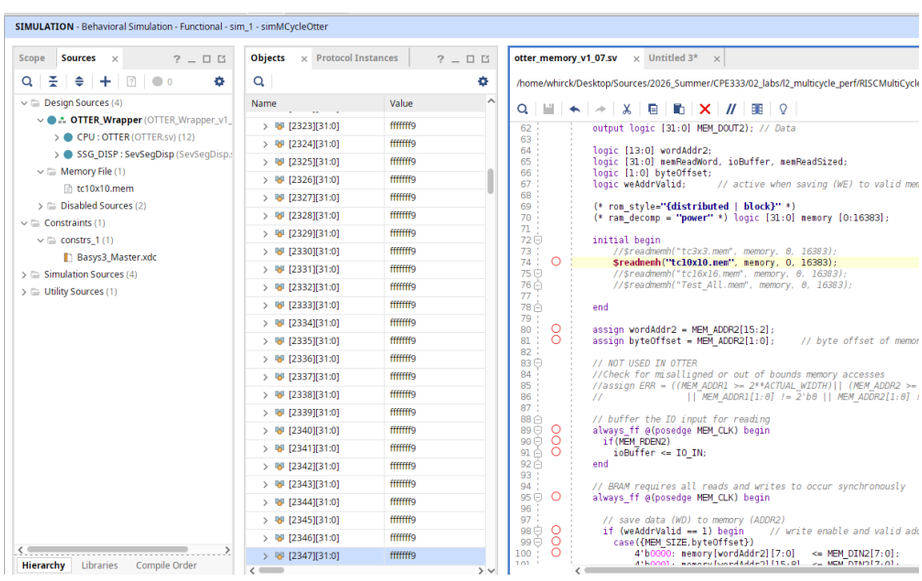


Figure 5: Results Matrix "C" before instruction execution.

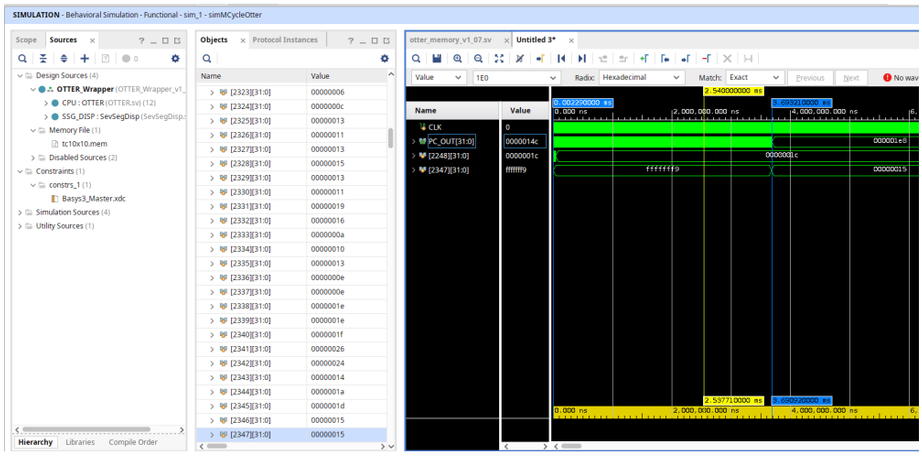


Figure 6: Waveform and Timing of TC 10x10 mat

$$
\begin{aligned}
t_{execution} &= t_{end} - t_{start} \\
&= 3.69321ms - 0.00229ms \\
&= 3.69092ms
\end{aligned}
$$

$$
\begin{aligned}
n_{CLK_Cycles} &= \frac{t_{execution}}{T_{CLK}} \\
&= \frac{3.69092ms}{10ns} \\
&= 369,092
\end{aligned}
$$

TC 16x16

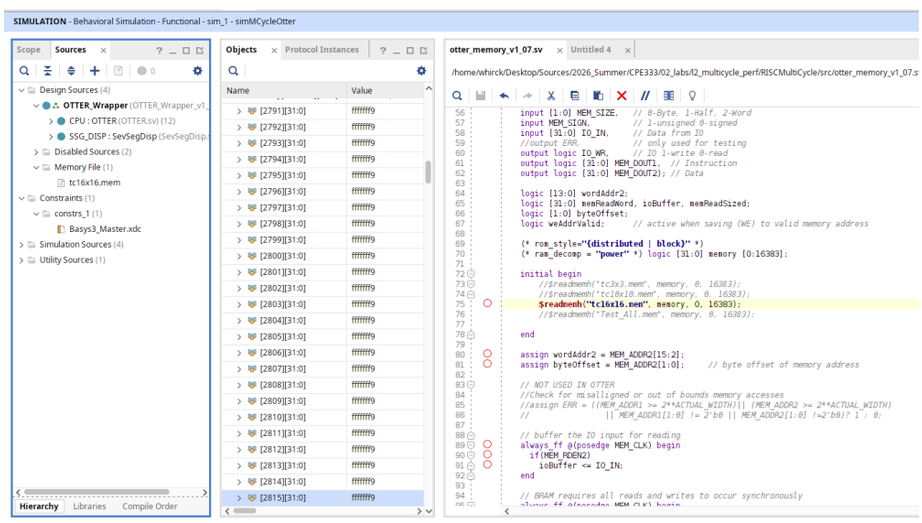


Figure 7: Results Matrix "C" before instruction execution.

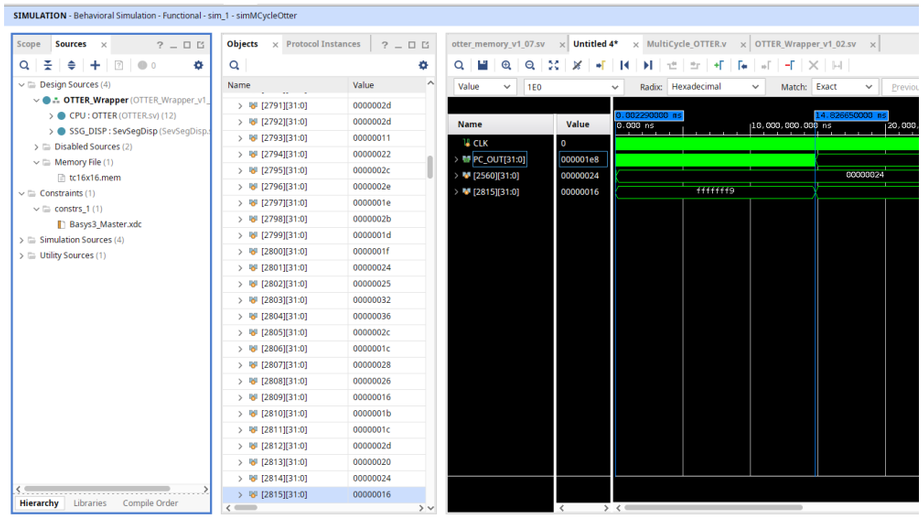


Figure 8: Waveform and Timing of TC 16x16 mat

$$
\begin{aligned}
t_{execution} &= t_{end} - t_{start} \\
&= 14.82665ms - 0.0022900ms \\
&= 14.82436ms
\end{aligned}
$$

$$
\begin{aligned}
n_{CLK_Cycles} &= \frac{t_{execution}}{T_{CLK}} \\
&= \frac{14.82665ms}{10ns} \\
&= 1,482,436
\end{aligned}
$$

MatrixMult Summary

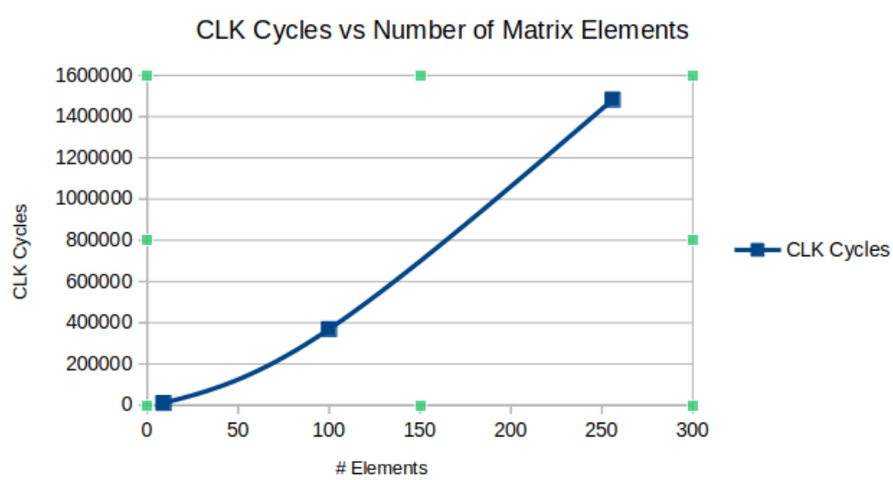


Figure 9: CLK Cycles vs. Number of Elements in the Matrix

Power and utilization are the same for each matrix testcase.

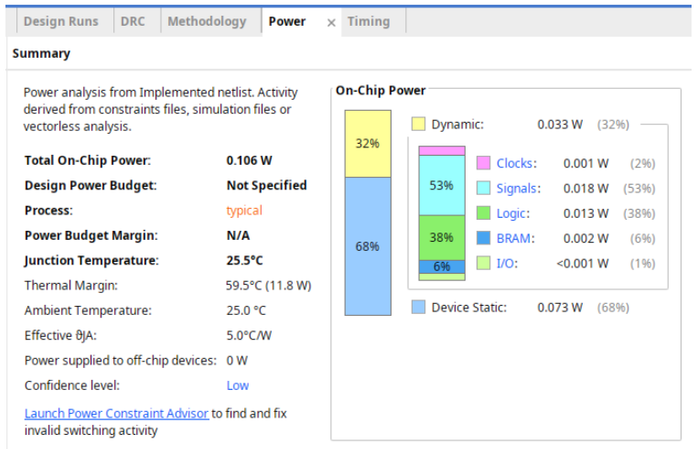
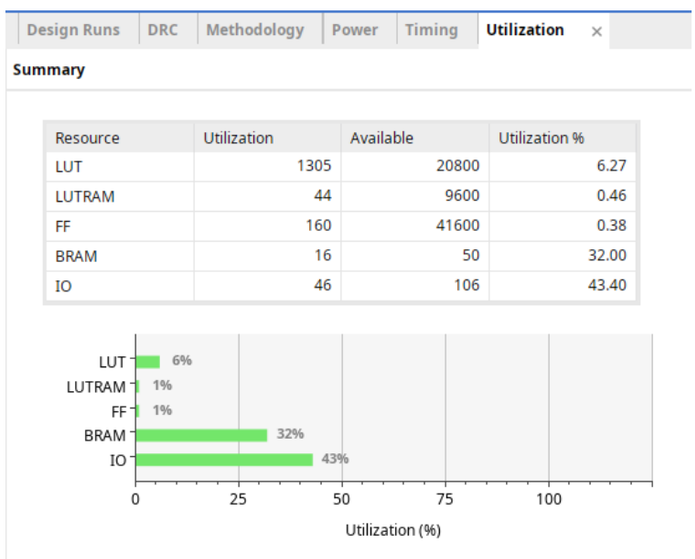


Figure 10: Power Summary



Resource	Utilization	Available	Utilization %
LUT	1305	20800	6.27
LUTRAM	44	9600	0.46
FF	160	41600	0.38
BRAM	16	50	32.00
IO	46	106	43.40

Figure 11: Utilization Summary

0.2 Testall Performance

0.2.1 Procedure

1. Load the testall mem file onto the multicycle otter. It should be noted that Diego's testall program worked well such that the Basys3 board also gave LED and sseg indicators that the system was working.

2. Capture the same waveforms, cpu execution time, power, and area utilization.

0.2.2 Results

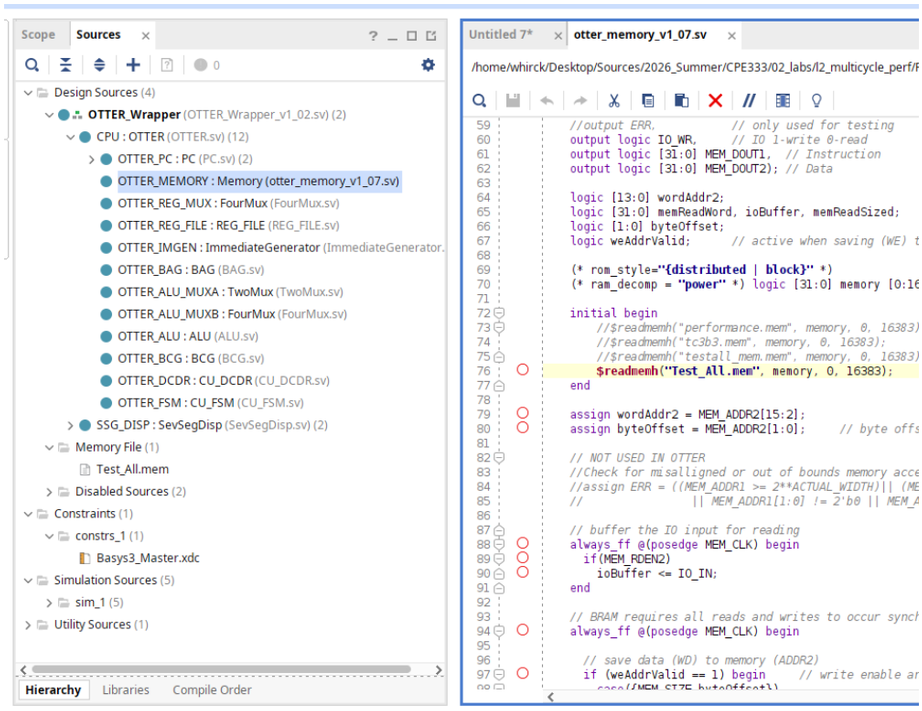
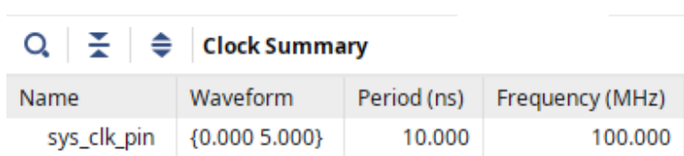


Figure 12: Uploading the testall memory

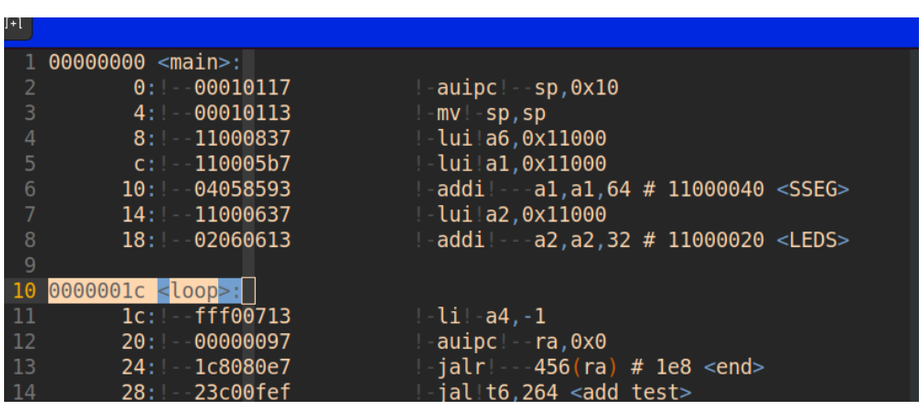
Timing



Clock Summary

Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	10.000	100.000

Figure 13: Clock Speed



```
00000000 <main>:
       0:	00010117	auipc	sp,0x10
       4:	00010113	mv	sp,sp
       8:	11000837	lui	a6,0x11000
       c:	110005b7	lui	a1,0x11000
      10:	04058593	addi	a1,a1,64 # 11000040 <SSEG>
      14:	11000637	lui	a2,0x11000
      18:	02060613	addi	a2,a2,32 # 11000020 <LEDS>

0000001c <loop>:
      1c:	fff00713	li	a4,-1
      20:	00000097	auipc	ra,0x0
      24:	1c8080e7	jalr	456(ra) # 1e8 <end>
      28:	23c00fef	jal	t6,264 <add_test>
```

Figure 14: Target Instruction Address for Timing Intervals in **Test_All_Debug.txt**

The *loop* label was used for measurement because the testall program returns to this after running all tests.

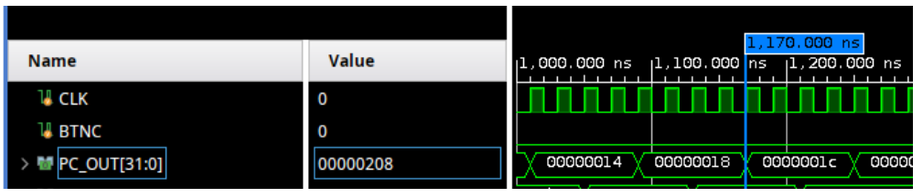


Figure 15: Measured Start Timestamp

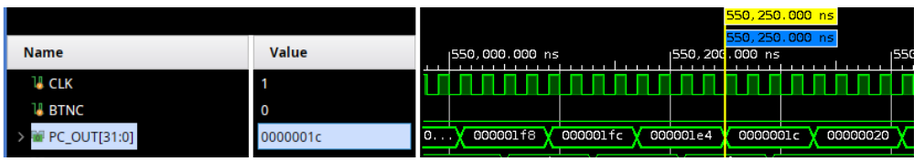


Figure 16: Measured End Timestamp

$$
\begin{aligned}
t_{execution} &= t_{end} - t_{start} \\
&= 550,250ns - 1,170ns \\
&= 549,080ns
\end{aligned}
$$

$$
\begin{aligned}
n_{CLK_Cycles} &= \frac{t_{execution}}{T_{CLK}} \\
&= \frac{549080ns}{10ns} \\
&= 54,908
\end{aligned}
$$

Note that the number above is not the number of instructions executed.

Other Metrics

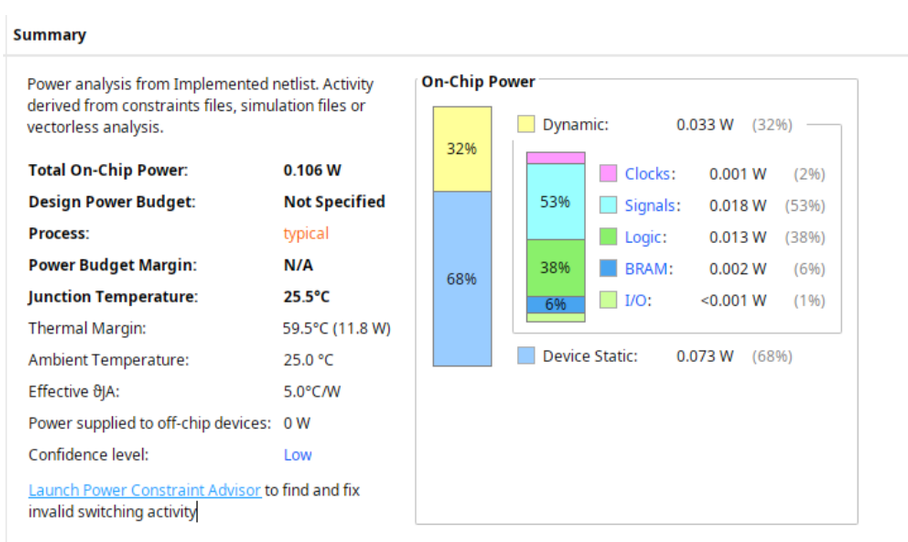
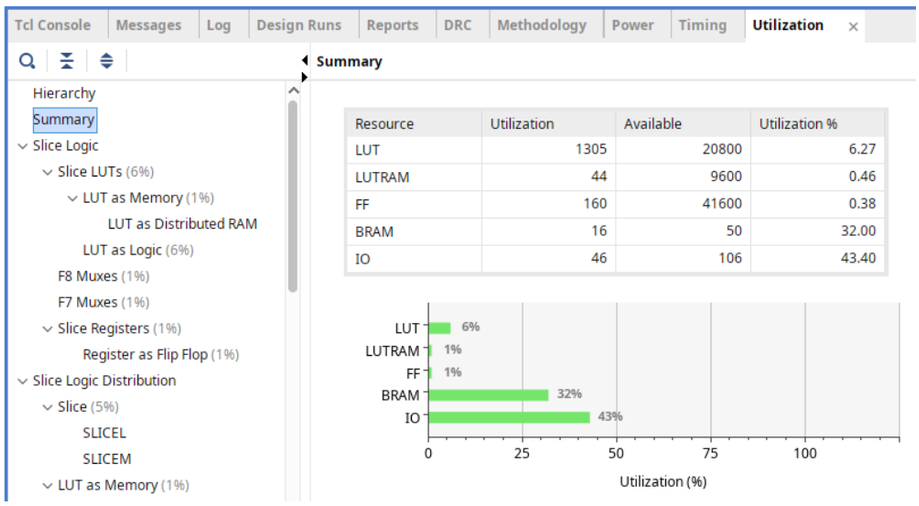


Figure 17: Power Summary of Testall on Basys3



Resource	Utilization	Available	Utilization %
LUT	1305	20800	6.27
LUTRAM	44	9600	0.46
FF	160	41600	0.38
BRAM	16	50	32.00
IO	46	106	43.40

Figure 18: Utilization Summary of Testall

0.3 Notes

0.3.1 RARS

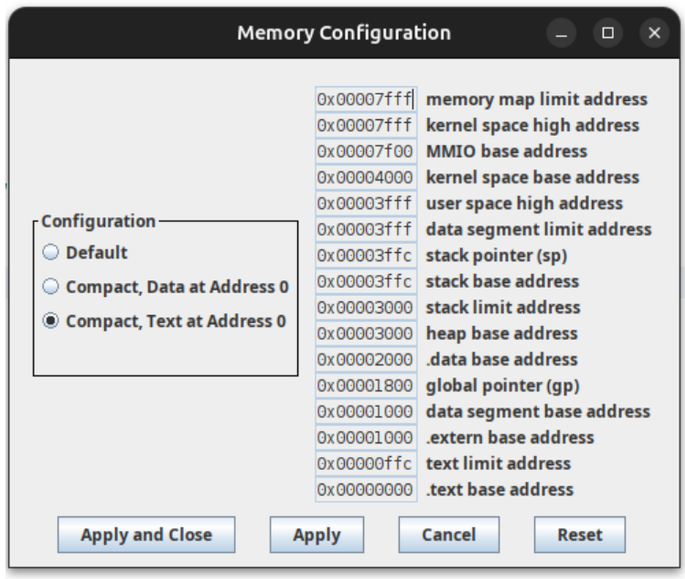


Figure 19: RARS Memory Config Settings

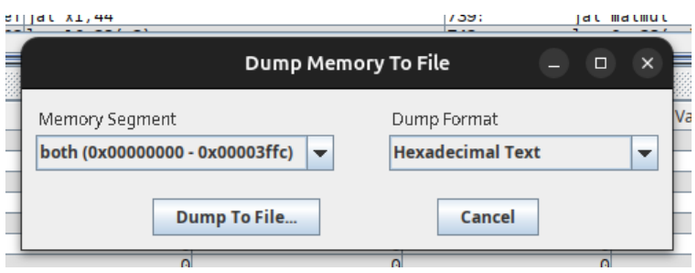


Figure 20: RARS Memory Dump Config

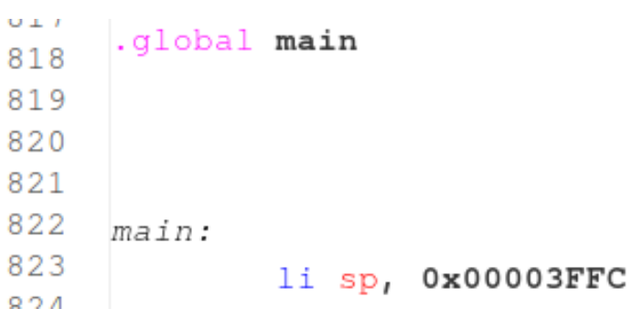


Figure 21: Stack Pointer set after Data Segment